

AN2DL - First Challenge Report

Budino Lovers

Carlo Bonandrini, Simone Decio, Jonel Relucio, Pietro Filippo Schgor

275682, 308376, 259768, 258727

November 17, 2025

1 Introduction

In this challenge, a collection of *multivariate time series* was given. Each sequence corresponds to a single subject and consists of 160 measurements taken at different time instants from several input channels. The objective is to perform *time series classification* to **predict the true pain status of the subjects**, classifying them into one of three categories: *no pain*, *low pain*, and *high pain* using models based on *Recurrent Neural Networks* architectures, **optimizing for the resulting F1-score**.

2 Problem Analysis

A *training dataset* of 661 labeled subjects and a *test dataset* of 1323 unlabeled subjects was provided. Each sample contains temporal dynamics of body joints and pain perception rule-based surveys. The features can be divided into three types:

- *Joint curves*: continuous measurements of body joint angles (neck, elbow, knee, etc.) over time for 31 joints.
- *Pain Surveys*: 4 simple rule-based sensor aggregations that estimate perceived pain (taking values: 0, 1, 2).
- *Subject characteristics*: 3 categorical variables that identify whether the subject is missing any limbs or eyes.

After data inspection, joint measurement curves exhibiting very low variance or showing strong correlation (> 0.8) with other joints were discarded, as they were contributing minimal or redundant information.

Furthermore, given the different distributions of values of each feature, a **min-max normalization** was applied to each joint.

3 Method

The *autocorrelation values* for each joint curve were analyzed and were all below 0.25 for every lag. This indicated minimal seasonality. Consequently, a medium-sized window was selected and a simple **bidirectional LSTM** [2] model was used. The model was then optimized using **Cross Entropy** as the loss function.

Noticing that the training dataset was really imbalanced with respect to the labels, with a much higher proportion of labels of type *no_pain*, a set of weights inversely proportional to the cardinality of each set was provided to the loss function to give more importance to the underrepresented categories.

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C w_c y_c \log(p_c) \quad (1)$$

The pipeline was structured as such: the training dataset was first split into training and validation

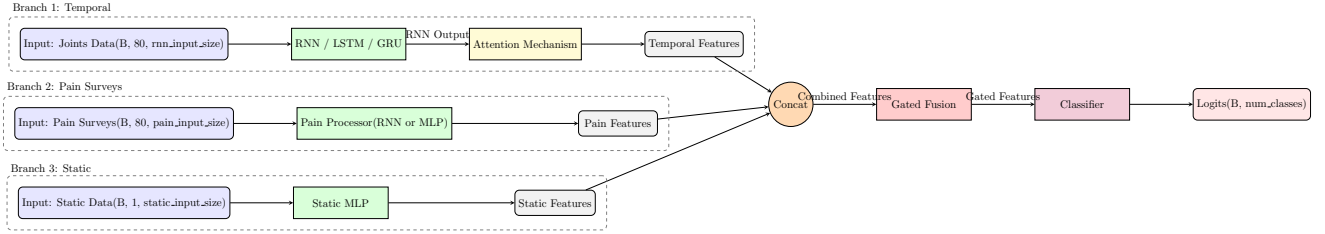


Figure 1: The final hybrid classifier architecture with three distinct branches for processing joints, pain surveys, and static subject data.

set with a 80-20 split and the batches were produced by taking a fixed window on the samples and moving it through the dataset with a distance set by the **stride** parameter. After this, hyperparameter tuning was performed. This was done using a **Bayesian Search** [3] approach, with each combination of hyper-parameters evaluated using **k-fold cross validation** and selected based on the resulting F1 score. This approach tries to estimate the importance of each hyperparameter to reduce the search space, as opposed to a simpler approach like **Grid Search** which tries all possible combinations. After finding the best combination, two approaches were used for retraining on the full dataset: the first one stopped training based on the epoch where the predicted label distribution on the test set matched the training distribution, and a second one by ensembling multiple full dataset trainings to account for stochasticity introduced by different starting random seeds.

4 Experiments

The majority class of the training labels was chosen as the baseline model, used to evaluate all others built afterwards. This gave a **baseline** F1-score of 0.6745 on the test set. In all models trained in the following paragraphs, the *AdamW Optimizer* was used. The earlier models were based on a simple architecture, where all features were directly fed to a single torch *rnn_module* of varying size, of type **LSTM** and the output passed through a fully-connected layer for calculating the logits corresponding to each label. This approach resulted in higher scores from the baseline, but still, it was clear that the logic was too simple. Not all features were in fact time-varying, so the static ones (the subject characteristics) were separated into a new

branch with a fully-connected layer that was then merged with the output of the *rnn_module* before the classifier. An attention mechanism was added to the output of the *rnn_module* of the joint features to select the best time steps to use in the later layers. The *pain surveys* features were also separated in their own branch with a *rnn_module* and merged with the others. This model, even with all its shortcomings, was the one that performed best during the whole competition, often reaching F1-scores between 0.94 and 0.95. [Figure 1]

After this, a more thoughtful approach was taken in the structuring and preprocessing of the model and underlying data. **Embedding layers** were the first addition to this architecture, first introduced on the *pain surveys*, giving the model the ability to find different representations of the underlying data in higher dimensions, therefore modifying the actual distance between each possible value of the feature. An embedding of the window index was also added during this phase. Analysis of their actual representations in higher dimensions showed few improvements in the separation of the values or actual performance, so this approach was scrapped.

A **1D-convolutional** layer was also introduced to preprocess the time-varying inputs to encode small local patterns in each series, while the *rnn_module* was charged with finding patterns in the medium to long term. This approach showed no improvements, so it was removed.

Feature engineering was also performed to extract and feed the network additional information, reducing the complexity the model had to learn by itself. As the joint measurements were angles, their first and second **derivatives** were added as input to capture velocity and acceleration, which

Table 1: Comparison between the baseline model and the best model

Model	F1-Score	Precision	Recall
Baseline	0.6741	0.5975	0.7730
Custom Model	0.9567 $\pm$ 0.0091	0.9584 $\pm$ 0.0095	0.9576 $\pm$ 0.0092

could contain useful patterns, though in practice this proved largely redundant. A more promising representation was then explored: the Spectrogram, which was fed into a Conv2D network, expecting to theoretically improve performance by capturing how the signal’s frequency evolves over time. This was immediately scrapped as the performance was suboptimal.

Monitoring gradient norms with **Tensorboard** during training helped identify issues, leading to the implementation of **gradient clipping**. This addition helped stabilize the gradients and prevented them from exploding.

To avoid the risk of overfitting as model complexity increases, the addition of regularization terms was fundamental. A **dropout rate** of 0.5 and an $L2 = 1e^{-3}$ **Ridge Regression** was used.

5 Results

The final refined architecture achieved an **F1-score of 0.9501 on the official test set**.

This is a noticeable improvement compared to the score of 0.6745 received with the baseline model. The improvements derived mainly from architectural separation of feature types, careful hyperparameter optimization, and loss reweighting for class imbalance. Models relying heavily on spectrograms or embeddings performed less consistently, suggesting that the temporal structure of the original signals was sufficiently captured by the RNN-based architecture. A table including the performance of the model compared to the baseline is included in Table 1.

6 Discussion

One of the main problems in this challenge was the disparity in the size of the training set compared to

the test set, which is almost double the size. Even so, given this limitation, the model is able to generalize quite well and doesn’t overfit. The model still has a lot of limitations given that it was abandoned in the initial stages of the challenge, replaced by fancier models, and then reworked in the later stages to try to push for the highest score possible. First of all it lacks some feature engineering from the later discarded models to enrich the information that the model receives, reducing the required size of the model if done correctly. The model is also quite large for a dataset of relatively small size, comprised in fact of 1,090,248 total parameters. Some hyper-parameters also need more tuning, but given the low availability of compute performance it was necessary to stop the optimization given the time constraints.

Also during these final stages of the challenge, the tuning of the model was based on the assumption that the distribution of the labels in the test set were similar to that of the training set. This assumption can’t be used in real world scenarios, but given the nature of the challenge at the time it seemed like the best choice.

7 Conclusions

The final model reaches a F1 score of 0.9501 on the test dataset, which is a solid result but still quite distant to the state of the art models found in the competition, reaching F1-scores higher than 0.9700. There is definitely space for future improvements, which may include:

- data augmentation for minority classes (*low pain and high pain*) via scaling and noise injection.
- Regularization and architecture simplification to reduce parameter count.
- implement a better hybrid CNN-RNN model.

References

- [1] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [2] R. C. Staudemeyer and E. R. Morris. Understanding long short-term memory recurrent neural networks – a tutorial-like introduction. *arXiv preprint arXiv:1909.09586*, 2019.
- [3] R. Turner, D. Eriksson, M. McCourt, J. Kiili, E. Laaksonen, Z. Xu, and I. Guyon. Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. *CoRR*, abs/2104.10201, 2021.